

Stellar Fusion — Architecture Spec

Working title. A 3D, sphere-based fusion puzzle game. Spiritual successor to 2048, set on a soccer-ball topology, governed by real stellar nucleosynthesis rules.

This document is a handoff spec for engineering. MVP scope is load-bearing (Section 13). Everything else is implementation detail.

0. Design decisions (resolved)

Question	Resolution
Geometry	Truncated icosahedron (12 pentagons + 20 hexagons, 32 faces total)
Merge mechanic	Drag-to-slide with per-element inertia (heavier = shorter slide)
Physics fidelity	Hybrid. Simplified 2-to-1 H fusion; alpha-capture chain for heavies; Be-8 unstable intermediate for He→C transition (with decay mechanic)
Game loop	Turn-based (MVP). Real-time / time-attack deferred to v2
Spawn cadence	One hydrogen on every committed move (drag that lands)
Pentagons	CNO cycle catalyst sites. H on pentagon self-fuses to He
Starting mass	Random 1–30 M _☉ , no player choice in MVP
End states	White dwarf, neutron star, black hole, jammed
Narrative copy	Stripped for MVP. Pure mechanics
Audio	Synthesized via Web Audio. No assets

Prior art

A flat 4x4 grid 2048-variant called "Stellar Fusion" / "Fe[26]" exists at dimit.me (Dimitar Dimitrov & Kevin O'Connor, RPI class project, based on Cirulli's open-source 2048). Their fusion chain is more detailed than ours (17 elements including Deuteron, He-3, Be-7, S, Ar, Ca, Ti, Cr, Fe-52, Ni-56). Their decay mechanic informs ours.

We differentiate via:

- 3D sphere topology (theirs is flat grid)
- Mass-based variable endings (theirs has one ending)
- Pentagon CNO catalyst sites (truncated icosahedron geometry)
- Phase system tied to stellar lifecycle
- Per-element slide inertia

Working title to be chosen; "Stellar Fusion" is taken. Candidates: Nucleosynth, Triple Alpha, Bethe, Iron Bridge, 56.

1. Tech stack

Layer	Choice	Rationale
Framework	React 18	Standard, ecosystem
Build	Vite	Fast HMR, minimal config
3D	Three.js via @react-three/fiber	Declarative R3F is easier to reason about than raw Three.js
3D helpers	@react-three/drei	OrbitControls, Html overlay, Stats
State	Zustand	No boilerplate, fits game loop, hooks-native
Styling	Tailwind	Utility-first, no component lib needed
Audio	Web Audio API (native)	Zero deps, synthesized tones
Persistence	localStorage	Resume game, no backend
Language	TypeScript	Type safety for the rules engine

No backend. No accounts. Single-player only. No analytics in MVP.

Explicitly rejected: Redux (overkill), Tone.js (overkill for simple synth), shadcn (overkill), any UI component library.

2. Folder structure

```
stellar-fusion/
├─ src/
│   ├─ main.tsx                # entry point
│   ├─ App.tsx                 # root component
│   └─ game/
│       ├─ state.ts            # zustand store
│       ├─ elements.ts         # element table (colors, inertia, pitches)
│       ├─ rules.ts            # merge rules engine
│       ├─ phases.ts           # phase transitions
│       ├─ endgame.ts          # win/lose determination
│       ├─ spawn.ts            # hydrogen rain logic
│       └─ types.ts            # shared types
│   └─ geometry/
│       ├─ truncatedIcosahedron.ts # face data + adjacency graph
│       ├─ projection.ts        # tangent-plane math
│       └─ slide.ts             # tile slide pathfinding
│   └─ three/
│       ├─ Scene.tsx           # canvas root
│       ├─ Sphere.tsx          # the playable sphere
│       ├─ Tile.tsx            # single face
│       ├─ Background.tsx      # starfield
│       └─ Controls.tsx        # orbit + drag gating
│   └─ ui/
│       ├─ HUD.tsx             # phase, mass, element legend
│       ├─ EndScreen.tsx
│       └─ StartScreen.tsx
│   └─ audio/
│       ├─ synth.ts            # merge tones
│       └─ ambient.ts          # space drone
│   └─ utils/
│       └─ math.ts             # vec3 helpers
├─ public/
├─ index.html
├─ vite.config.ts
├─ tailwind.config.js
├─ tsconfig.json
└─ package.json
```

3. Core data model

```
// src/game/types.ts

export type ElementSymbol = 'H' | 'He' | 'C' | 'O' | 'Ne' | 'Mg' | 'Si' | 'Fe';

export interface Element {
  symbol: ElementSymbol;
  atomicNumber: number;
  displayName: string;
  color: string;           // hex, biased toward stellar spectrum
  slideDistance: number;   // max faces traversable per drag (0 = immovable)
  pitch: number;           // Hz, for merge synth
}

export interface Tile {
  faceId: number;          // index into face array
  element: ElementSymbol;
  spawnedAtTurn: number;   // for animations
}

export type FaceShape = 'pentagon' | 'hexagon';

export interface Vec3 { x: number; y: number; z: number; }

export interface Face {
  id: number;
  shape: FaceShape;
  center: Vec3;            // unit sphere position
  vertices: Vec3[];        // local geometry (5 or 6)
  neighbors: number[];     // adjacent face ids
  tangentFrame: {          // for drag projection
    u: Vec3;
    v: Vec3;
    n: Vec3;               // outward normal
  };
}

export type Phase = 'main_sequence' | 'red_giant' | 'supergiant' | 'collapse';

export type EndState =
  | 'white_dwarf'
  | 'neutron_star'
  | 'black_hole'
  | 'failed_collapse'      // iron formed at wrong mass
  | 'jammed';              // sphere full, no merges possible
```

```
export interface GameState {  
  // immutable per run  
  starMass: number;  
  faces: Face[];  
  
  // mutable  
  tiles: Map<number, Tile>; // faceId -> tile  
  turn: number;  
  phase: Phase;  
  elementCounts: Record<ElementSymbol, number>;  
  
  // ui-only  
  selectedFaceId: number | null;  
  isAnimating: boolean;  
  endState: EndState | null;  
}
```

4. Geometry layer

4.1 Truncated icosahedron

Standard construction. 60 vertices, 32 faces, 90 edges.

Algorithm:

1. Start with regular icosahedron (12 vertices using golden-ratio coordinates).
2. For each edge, place two truncation points at $1/3$ and $2/3$ along it. These 60 points become the vertices of the truncated icosahedron.
3. **Pentagons** form around each original icosahedron vertex — the 5 nearest truncation points.
4. **Hexagons** replace each original triangular face — its 3 corners are cut off, leaving a hexagon.
5. For each face: compute centroid (face center), sort vertices counterclockwise around the centroid, compute outward normal, build tangent frame.

Output: `Face[]` of length 32, with adjacency populated.

This data is static. Compute once at game start, cache in store.

4.2 Adjacency graph

Each pentagon has 5 neighbors (all hexagons). Each hexagon has 6 neighbors (3 pentagons + 3 hexagons — verify with tests). Build a symmetric adjacency map. Used for:

- Merge detection (same-element neighbor)
- Triple-alpha detection (3 He all mutually adjacent forming a triangle)
- Slide pathfinding

4.3 Tangent frame per face

Each face has an orthonormal basis (u, v, n):

- n = face center direction (outward normal on unit sphere)
- u = arbitrary perpendicular to n (e.g., `normalize(cross(n, world-up))`)
- v = `cross(n, u)`

Used to project 2D screen drags into 3D directions on the sphere surface.

4.4 Slide direction resolution

When the player drags from face A in screen-space direction `d_screen`:

```
function resolveSlideTarget(
  fromFace: Face,
  dragWorld: Vec3,           // drag vector projected to world space
  faces: Face[]
): number | null {
  // Project drag onto face A's tangent plane
  const dTangent = projectToPlane(dragWorld, fromFace.tangentFrame.n);
  if (length(dTangent) < EPSILON) return null; // drag was purely radial

  let bestId = -1;
  let bestDot = -Infinity;

  for (const neighborId of fromFace.neighbors) {
    const toNeighbor = subtract(faces[neighborId].center, fromFace.center);
    const projected = projectToPlane(toNeighbor, fromFace.tangentFrame.n);
    const dot = dotProduct(normalize(projected), normalize(dTangent));
    if (dot > bestDot) {
      bestDot = dot;
      bestId = neighborId;
    }
  }

  return bestId;
}
```

```
}
```

4.5 Slide execution

Tile slides up to its `slideDistance` faces in the resolved direction. Each step:

1. If next face in current direction is empty → move tile there.
2. If next face has same element + merge rule exists → STOP, trigger merge.
3. If next face is occupied with different element (or no merge rule) → STOP (blocked).
4. If reached `slideDistance` limit → STOP.

Direction must be re-resolved at each face (since each face has its own tangent frame and neighbor set). Carry forward the original drag vector and reproject at each step.

5. Rendering layer (R3F)

5.1 Scene composition

```
// src/three/Scene.tsx
export function Scene() {
  return (
    <Canvas camera={{ position: [0, 0, 5], fov: 50 }}>
      <Background />
      <ambientLight intensity={0.3} />
      <pointLight position={[5, 5, 5]} intensity={1.5} />
      <Sphere />
      <Controls />
    </Canvas>
  );
}
```

5.2 Sphere component

32 separate meshes (one per face). Recommendation: stay with separate meshes for MVP — pointer events and per-tile animations are trivial. Optimize to instanced meshes only if profiling shows it matters.

```
// src/three/Sphere.tsx
export function Sphere() {
  const faces = useGameStore(s => s.faces);
```

```

const tiles = useGameStore(s => s.tiles);
const scale = useGameStore(s => phaseScale(s.phase));

return (
  <group scale={scale}>
    {faces.map(face => (
      <Tile key={face.id} face={face} tile={tiles.get(face.id)} />
    ))}
  </group>
);
}

```

5.3 Tile component

- Custom polygon geometry from face vertices
- Color from element table (or dark gray if empty)
- Element symbol rendered via `<Html>` overlay from drei (CSS-positioned text), or texture-baked
- `onPointerDown` / `onPointerMove` / `onPointerUp` for drag
- Pentagons get a subtle inner outline — communicates “different” without explicit UI

5.4 Background

Particle system: 800 random points on a far sphere (radius 50). Twinkle via shader uniform driven by `useFrame`. No nebulas, no parallax — just stars. Keep it simple.

5.5 Phase transitions (visual)

On phase change, animate `group.scale` from current → target over 2 seconds (easeOutQuad). Targets: 1.0 (main sequence), 1.3 (red giant), 1.6 (supergiant), 0.4 (collapse). Camera does not move — sphere appears to grow/shrink in frame.

Optional polish (post-MVP): surface color shift across the sphere during red giant phase to communicate temperature change.

6. Input / interaction

6.1 Input state machine

IDLE

- └ pointer down on tile → TILE_SELECTED
- └ pointer down on background → ORBIT_ROTATION

TILE_SELECTED

- └ pointer move > THRESHOLD → DRAGGING
- └ pointer up → IDLE (no-op for MVP; reserve for tap actions later)

DRAGGING

- └ pointer up → EXECUTE_SLIDE → ANIMATING

ANIMATING

- └ animation complete → check merge → maybe spawn → check endgame → IDLE

ORBIT_ROTATION

- └ pointer up → IDLE

OrbitControls **must** be disabled in TILE_SELECTED, DRAGGING, ANIMATING states. Re-enabled in IDLE.

6.2 Drag projection

When the user drags, take the screen-space delta, multiply by camera basis to get a world-space vector, then pass to `resolveSlideTarget`. Don't try to do this purely in 2D — the sphere curves and you need real world directions.

6.3 Animation

- **Slide:** tile center lerps face A → face B over 150ms. For chained slides (multiple faces), chain the lerps with no pause between segments.
- **Merge:** 200ms. Two source tiles converge to the merge location, scale-pulse to 1.2× then 0, replaced by product tile spawning at scale 0 → 1. Particle burst (10–20 short-lived points in element color).
- **Spawn:** hydrogen fades in at scale 0 → 1 over 300ms after merge animation completes.

Use `useFrame` for per-frame interpolation. Framer Motion 3D is acceptable if engineer prefers declarative — your call.

7. Game logic

7.1 Element table

```
// src/game/elements.ts
```

```
export const ELEMENTS: Record<ElementSymbol, Element> = {
  H: { symbol: 'H', atomicNumber: 1, displayName: 'Hydrogen', color: '#ff6b6b'
  He: { symbol: 'He', atomicNumber: 2, displayName: 'Helium', color: '#feca57'
  C: { symbol: 'C', atomicNumber: 6, displayName: 'Carbon', color: '#48dbfb'
  O: { symbol: 'O', atomicNumber: 8, displayName: 'Oxygen', color: '#1dd1a1'
  Ne: { symbol: 'Ne', atomicNumber: 10, displayName: 'Neon', color: '#ff9ff3'
  Mg: { symbol: 'Mg', atomicNumber: 12, displayName: 'Magnesium', color: '#a29bfe'
  Si: { symbol: 'Si', atomicNumber: 14, displayName: 'Silicon', color: '#dfe6e9'
  Fe: { symbol: 'Fe', atomicNumber: 26, displayName: 'Iron', color: '#636e72'
};
```

Pitches are spaced by perfect fourths (220, 277, 330, 392, ...) so merge tones land in a pleasant scale.

Iron has `slideDistance: 0` — it cannot be moved. This is mechanically why iron jams the sphere if you over-produce it. Honors the physics: iron is the gravitational dead end.

7.2 Merge rules

```
// src/game/rules.ts
```

```
export type MergeRule = {
  inputs: ElementSymbol[];
  output: ElementSymbol;
  pattern: 'pair' | 'triangle' | 'pair_alpha';
  requiresPentagon?: boolean;
};

export const MERGE_RULES: MergeRule[] = [
  // Hydrogen burning (simplified from 4-to-1 to 2-to-1 for playability)
  { inputs: ['H', 'H'], output: 'He', pattern: 'pair' },

  // CNO cycle shortcut: H on a pentagon self-fuses
  { inputs: ['H'], output: 'He', pattern: 'pair', requiresPentagon: true },

  // Triple-alpha process: 3 He → C (skipping unstable Be-8 intermediate)
  { inputs: ['He', 'He', 'He'], output: 'C', pattern: 'triangle' },

  // Alpha-capture chain (real)
  { inputs: ['C', 'He'], output: 'O', pattern: 'pair_alpha' },
  { inputs: ['O', 'He'], output: 'Ne', pattern: 'pair_alpha' },
  { inputs: ['Ne', 'He'], output: 'Mg', pattern: 'pair_alpha' },
  { inputs: ['Mg', 'He'], output: 'Si', pattern: 'pair_alpha' },
```

```
// Silicon burning (simplified — real chain is Si → Ni-56 → Fe-56)
{ inputs: ['Si', 'Si'], output: 'Fe', pattern: 'pair' },
];
```

7.3 Merge detection (post-slide)

After a tile lands on a face, check in this order:

1. **Triangle (triple-alpha)**: only if landed element is He. Find pairs of He neighbors that are also adjacent to each other. If found → triple-alpha to C.
2. **Pair (same element)**: scan neighbors for matching element with a defined `pair` or `pair_alpha` rule.
3. **Pentagon shortcut**: if landed element is H and landed face is a pentagon → self-fuse to He.

Returns `MergeRule` | `null`. Caller applies the rule:

- Remove all input tiles.
- Place output tile on the landed face's position.
- Increment `elementCounts[output]`.
- Trigger merge animation.
- Play merge synth.

7.4 Why these rules and not others

The physics-honest version of H-burning is 4 protons → He, which would require finding 4 H tiles in a small cluster. Playtests of similar mechanics show this feels punishing and confusing. The 2-to-1 simplification preserves the *feel* of 2048's doubling cadence while keeping the rest of the chain physically real.

The triple-alpha is honored as a triangle requirement because it's mechanically distinctive — players will feel the regime change from "merge pairs" to "find triangles" when entering the He → C transition. That regime change *is* the red giant phase, which is exactly the right narrative beat.

8. Phase system

```
// src/game/phases.ts

interface PhaseRule {
  phase: Phase;
  triggers: (state: GameState) => boolean;
  visualScale: number;
  hSpawnRate: number;      // hydrogens per committed move
  unlocksElements: ElementSymbol[];
}

export const PHASES: PhaseRule[] = [
  {
    phase: 'main_sequence',
    triggers: () => true,           // default
    visualScale: 1.0,
    hSpawnRate: 1,
    unlocksElements: ['H', 'He'],
  },
  {
    phase: 'red_giant',
    triggers: s => s.elementCounts.He >= 8,
    visualScale: 1.3,
    hSpawnRate: 1,
    unlocksElements: ['H', 'He', 'C', 'O'],
  },
  {
    phase: 'supergiant',
    triggers: s => s.elementCounts.C >= 4 && s.starMass >= 8,
    visualScale: 1.6,
    hSpawnRate: 2,
    unlocksElements: ['H', 'He', 'C', 'O', 'Ne', 'Mg', 'Si'],
  },
  {
    phase: 'collapse',
    triggers: s => s.elementCounts.Fe >= 1,
    visualScale: 0.4,
    hSpawnRate: 0,               // star is dyi
    unlocksElements: [],
  },
];
```

After every merge, scan phases in reverse order, set current phase to the first whose `triggers` returns true. Trigger sphere puff animation if phase changed.

`unlocksElements` is informational for the HUD legend — it tells the player what they can theoretically reach in this phase. Doesn't restrict gameplay.

9. Spawn system (hydrogen rain)

```
// src/game/spawn.ts

export function spawnHydrogen(state: GameState): void {
  const phase = currentPhaseRule(state);
  const emptyFaces = state.faces.filter(f => !state.tiles.has(f.id));

  for (let i = 0; i < phase.hSpawnRate; i++) {
    if (emptyFaces.length === 0) return;
    const idx = Math.floor(Math.random() * emptyFaces.length);
    const target = emptyFaces[idx];
    state.tiles.set(target.id, {
      faceId: target.id,
      element: 'H',
      spawnedAtTurn: state.turn,
    });
    emptyFaces.splice(idx, 1);
  }
}
```

Called after every **committed move** — any drag that resulted in a tile moving, whether or not it produced a merge. Reasoning: rewards efficiency. If you make a useless drag, you still get punished with new H. This preserves 2048's “every move costs you” pressure.

In collapse phase, spawn rate is 0. The star is dying. No new fuel.

Edge case: if sphere has 0 empty faces and spawn is called, no-op. Endgame check will catch the jam.

10. Endgame

```
// src/game/endgame.ts

export function checkEndState(state: GameState): EndState | null {
  // Jam: sphere full, no possible merges
  if (isJammed(state)) return 'jammed';

  // Iron formed → core collapse triggered
  if (state.elementCounts.Fe >= 1) {
    if (state.starMass < 1.4) return 'failed_collapse';
    if (state.starMass < 8) return 'white_dwarf';
  }
}
```

```

    if (state.starMass < 25)    return 'neutron_star';
    return 'black_hole';
  }

  // Low-mass natural end: enough O accumulated, H exhausted
  if (state.starMass < 8
      && state.elementCounts.O >= 4
      && state.elementCounts.H === 0) {
    return 'white_dwarf';
  }

  return null;
}

function isJammed(state: GameState): boolean {
  if (state.tiles.size < state.faces.length) return false;
  for (const [faceId] of state.tiles) {
    if (detectMerge(faceId, state)) return false;
  }
  return true;
}

```

End screen shows:

- End-state name and a one-line description
- Star mass at start
- Element counts as horizontal bars
- “Play again” button

No copy beyond facts in MVP. Personality added later.

11. Audio

11.1 Merge synth

```

// src/audio/synth.ts

let ctx: AudioContext | null = null;

export function initAudio(): void {
  if (!ctx) ctx = new AudioContext();
  if (ctx.state === 'suspended') ctx.resume();
}

```

```

}

export function playMerge(parent: ElementSymbol, child: ElementSymbol): void {
  if (!ctx) return;

  const parentPitch = ELEMENTS[parent].pitch;
  const childPitch = ELEMENTS[child].pitch;
  const now = ctx.currentTime;

  const osc = ctx.createOscillator();
  const gain = ctx.createGain();

  osc.type = 'sine';
  osc.frequency.setValueAtTime(parentPitch, now);
  osc.frequency.exponentialRampToValueAtTime(childPitch, now + 0.3);

  gain.gain.setValueAtTime(0, now);
  gain.gain.linearRampToValueAtTime(0.3, now + 0.02);
  gain.gain.exponentialRampToValueAtTime(0.001, now + 0.5);

  osc.connect(gain).connect(ctx.destination);
  osc.start(now);
  osc.stop(now + 0.5);
}

```

Each merge is a swooping tone from parent pitch up to child pitch. Heavier elements have higher pitches. Players learn the element hierarchy through their ears.

11.2 Ambient drone

Single low oscillator at 60 Hz + a fifth at 90 Hz, both fed into a lowpass filter, slight LFO modulation. Output gain ~0.05. Plays continuously after first user interaction.

11.3 Spawn tick

Soft click on hydrogen spawn. Single 1200 Hz sine, 20ms envelope. Output gain ~0.1. Easy to overdo — keep it quiet.

11.4 Browser autoplay constraint

Audio context cannot start without user interaction. Initialize on first click (start button). Cache `ctx` in module scope.

12. UI / HUD

Minimal. Tailwind, absolute-positioned divs over the canvas.

Top bar (left to right):

- Phase name + small phase icon (○ for main sequence, ● for red giant, ⊙ for supergiant)
- Star mass (e.g., "12.3 M $\odot$ ")
- Turn counter

Right rail:

- Element legend, vertical. Each row: symbol + color swatch + current count. Locked elements grayed out.

End screen:

- Full-canvas overlay with backdrop blur
- End-state name in large display type
- Element counts as horizontal bars
- "Play again" button

No tooltips, no tutorial, no settings menu in MVP.

13. MVP scope — load-bearing

In

- Truncated icosahedron sphere (32 faces)
- 8 elements: H, He, C, O, Ne, Mg, Si, Fe
- Drag-to-slide with per-element inertia
- Per-merge hydrogen spawn
- 4 phases: main sequence, red giant, supergiant, collapse
- 4 end states: white dwarf, neutron star, black hole, jammed
- Synthesized audio: merge tones, ambient drone, spawn tick
- Random star mass 1–30 M $\odot$ per run
- Pentagon CNO shortcut for H $\rightarrow$ He

- Phase-driven sphere puff visual

Out (v2+)

- R-process bonus round (gold, uranium from supernova/kilonova)
- Multi-layer playable sphere (onion shells)
- Time-attack / real-time mode
- Meta-progression, unlockable masses
- Custom star mass selection
- Narrative copy ("atoms in your body...")
- Mobile-specific UX (assume desktop first)
- Be-8 unstable intermediate (with decay timer)
- Failed-collapse penalty animation
- Binary / neutron-merger modes
- Tutorial / onboarding
- Settings menu / accessibility options
- Leaderboard / persistence beyond resume

Why these cuts

Each item in "Out" can be added without rearchitecting. The MVP exists to validate the core loop: drag-slide-merge-spawn-progress. If that loop isn't fun, none of the v2 features will save it. If it is fun, the v2 features are tasteful additions, not rescues.

14. Build & deploy

```
npm create vite@latest stellar-fusion -- --template react-ts
cd stellar-fusion
npm i three @react-three/fiber @react-three/drei zustand
npm i -D tailwindcss postcss autoprefixer @types/three
npx tailwindcss init -p
```

Vercel or Netlify. Static site, no backend. Bundle target < 500 KB gzipped (R3F + Three.js core is ~150 KB; rest is room).

15. Test strategy

Unit tests (Vitest)

- `geometry/truncatedIcosahedron.test.ts` : face count = 32, pentagon count = 12, hexagon count = 20, every pentagon has 5 hexagon neighbors, every hexagon has 3 hexagon + 3 pentagon neighbors, adjacency is symmetric.
- `geometry/slide.test.ts` : direction resolution returns the expected neighbor for canonical drag vectors.
- `game/rules.test.ts` : every merge rule produces correct output for fixture inputs. Triangle detection on known He triples.
- `game/endgame.test.ts` : end-state determination across the mass range.
- `game/phases.test.ts` : each phase trigger fires only when expected.

Manual playtest checklist

- On new game, sphere has ~5 hydrogens spawned
- Drag H across empty sphere — slides 8 faces or until blocked
- H + H adjacent → He merge, with sound and animation
- H on pentagon → self-fuses to He
- 3 He in a triangle → C merge
- C + He adjacent → O
- Phase transitions trigger and sphere puffs
- Sphere fills past valid merges → jam end screen
- 15 $M_{\odot}$ run that produces Fe → neutron star ending
- OrbitControls disabled during drag, re-enabled after
- Audio context resumes after first click (no console errors)
- Game state persists to localStorage and resumes on reload

16. Known risks

Risk	Severity	Mitigation
Drag direction resolution feels janky on tilted faces	High	Prototype drag + slide at step 4 of implementation order. Tune projection math against playtest before continuing.
32 tiles is too many — game drags before reaching He	Medium	Easy fix: drop to a dodecahedron (12 faces) for low-mass MVP. Trades pentagons-as-special for everything-is-pentagon, but proves the loop faster.
Pentagon shortcut is invisible to players	Medium	Add a subtle inner outline on pentagons. If still invisible, brief tooltip on first pentagon spawn.
Phase transition thresholds feel arbitrary	Medium	Externalize all thresholds to a config object. Tune via playtest, not code change.
Sphere occlusion confusing — tiles on far side hidden	Medium	Optional: render back-side tiles as 30%-opacity wireframes. Or auto-rotate the sphere to center the most recent merge action. Defer decision to first playtest.
Audio context blocked by browser autoplay policy	Low	Require explicit click on Start. Initialize and <code>resume()</code> audio context in that handler.
Iron jamming feels like a bug, not a feature	Medium	Telegraph via copy on end screen: "Iron cannot be fused. Your star's core has collapsed."

17. Implementation order

Build in this order. Each step is a working, testable artifact.

1. Truncated icosahedron geometry + neighbor graph. No game yet. Visualize as a colored mesh, verify adjacency by clicking and listing neighbors.
2. R3F scene rendering all 32 faces with placeholder colors. OrbitControls working.
3. Click selection — highlight on click, no drag yet.
4. Drag direction resolution → slide a single tile across an empty sphere.
5. Spawn N hydrogens on game start (random faces).

6. Merge detection + execution for $H + H \rightarrow He$ only. No animations yet.
7. Hydrogen rain on every committed move.
8. 🚦 **Feel-test checkpoint.** Playtest steps 1–7. Is the drag-slide-merge-spawn loop satisfying? If no, redesign before continuing. Do not skip this gate.
9. Full element chain + alpha-capture rules.
10. Triple-alpha (triangle detection) for $He \rightarrow C$.
11. Phase system + sphere puff animations.
12. End-state detection + end screen.
13. Pentagon CNO shortcut.
14. Audio: merge synth, then ambient, then spawn tick.
15. Polish pass: particles, animations, ambient color shifts, easing curves.

Steps 1–7 is roughly 3–5 days for a competent engineer. Steps 9–15 are about the same.
Total MVP: 2 weeks of focused work. Anything that delays should be cut from MVP.

18. Open questions for the engineer

Things that need decisions during implementation, not now:

1. **Polygon mesh strategy:** 32 separate meshes vs single buffer geometry with face groups. Start with separate meshes; profile if needed.
 2. **Element symbols on tiles:** Html overlay (CSS, crisp text) vs texture-baked (better integration but pixelated at zoom). Start with Html; revisit if it feels wrong.
 3. **Slide chain handling:** Should iron block sliding entirely, or should slides “pile up” against iron the way 2048 stacks against a wall? Current spec says block — playtest will tell.
 4. **Failed_collapse semantics:** Currently triggered if star mass < 1.4 and Fe forms. Is this actually achievable? May need rule adjustment.
 5. **First-merge tutorial:** zero-cost addition (one tooltip on first drag), but stripped from MVP. Add if playtests show confusion.
-

End of spec

Hand to engineering. Build steps 1–7 first. Feel-test. Decide whether to continue or redesign.